

MODULE 6

Indexing and Query Performance

How MongoDB finds documents, how to design indexes around real query shapes, and how to prove the result with `explain()`.

**COLLSCAN vs
IXSCAN**

Compound + ESR

Specialized indexes

`explain()` + covered

Costs + clean-up

LEARNING OBJECTIVES

What you will be able to do by the end of this module.



BY THE END OF THIS MODULE, YOU CAN

1. Explain how an index avoids a full scan
2. Create, list, hide and drop named indexes
3. Order compound fields with ESR
4. Choose multikey, unique, partial, sparse, TTL
5. Read explain() and build a covered query
6. Weigh selectivity, write cost and usage



MODULE ROADMAP



SIX QUESTIONS THIS MODULE ANSWERS

WHY

does a query read every document?

HOW

do I create and manage indexes?

WHICH

field order goes in an index?

WHICH

index type fits the query?

WHAT

does explain() really tell me?

WHEN

is an index not worth its cost?

KEY TAKEAWAY Design indexes around the queries the application really runs — then prove them with explain().

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 Index

A sorted structure of field values with pointers to documents, so MongoDB can skip the full scan.

2 COLLSCAN and IXSCAN

Plan stages: read every document, or walk the matching keys of one index.

3 Compound index

One index on several fields, sorted by the first field, then the next.

4 ESR guideline

Order compound fields Equality first, then Sort, then Range — a starting point, not a law.

5 Multikey index

An index on an array field: one index key for each array value.

6 explain()

Shows the plan MongoDB chose and, with executionStats, the keys and documents it examined.

7 Covered query

Filter and returned fields all come from one index, so no document is fetched.

8 Selectivity

How small a share of the collection a condition matches — sku is high, active is low.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

COLLSCAN

Collection Scan

I/O

Input/Output

RAM

Random Access Memory

CPU

Central Processing Unit

IXSCAN

Index Scan

SKU

Stock Keeping Unit

ESR

Equality, Sort, Range

JSON

JavaScript Object Notation

TTL

Time To Live

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 5 TO MODULE 6

Day 2 taught us to ask questions — Day 3 makes them fast.

MODULE 5 GAVE US

- ✓ Pipelines of \$match, \$group and \$sort
- ✓ Reports over orders and reviews
- ✓ \$lookup joins between collections
- ✓ Filter early to shrink the work
- ✓ Results checked against real data

NOW Index the query shapes Day 2 wrote — and prove each one with explain().



OUR PATH THROUGH MODULE 6

- 1 See why a query reads every document
- 2 Create, list, hide and drop indexes
- 3 Order compound fields with ESR
- 4 Pick a specialized index type
- 5 Read explain() and cover a query
- 6 Weigh selectivity, costs and mistakes

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

KEY TAKEAWAY Module 5 asked the questions; Module 6 makes MongoDB answer them without reading every document.

PART 1: HOW INDEXES WORK

Start with how MongoDB finds a document at all.



SO FAR

- ✓ Module 5 built pipelines over our data
- ✓ Every query so far ran without asking how

NEXT QUESTION

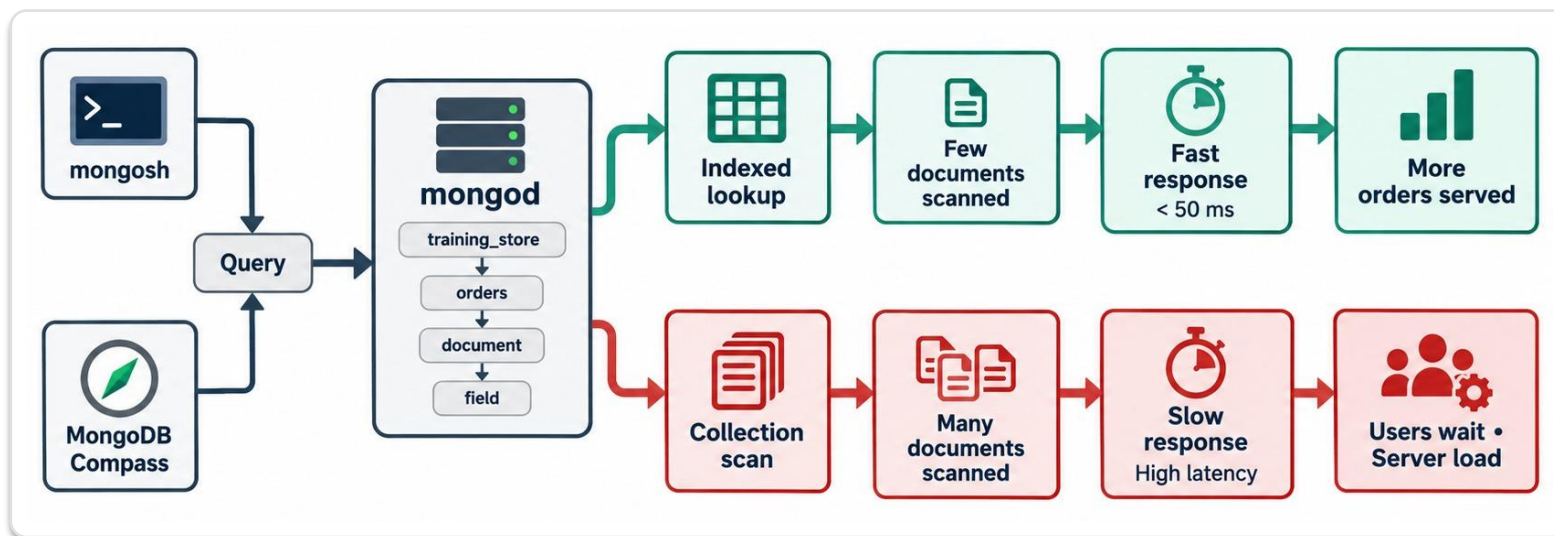
How does MongoDB find documents — and why does an index make it faster?

IN THIS PART

- 1 Why query performance matters
- 2 Collection scan versus index scan
- 3 What an index looks like inside
- 4 The `_id` index every collection has

WHY QUERY PERFORMANCE MATTERS

The same query: an indexed lookup or a full collection scan.



Indexed lookup

Few documents read — fast, predictable responses.

Collection scan

Every document read — the cost grows with the collection.

At scale

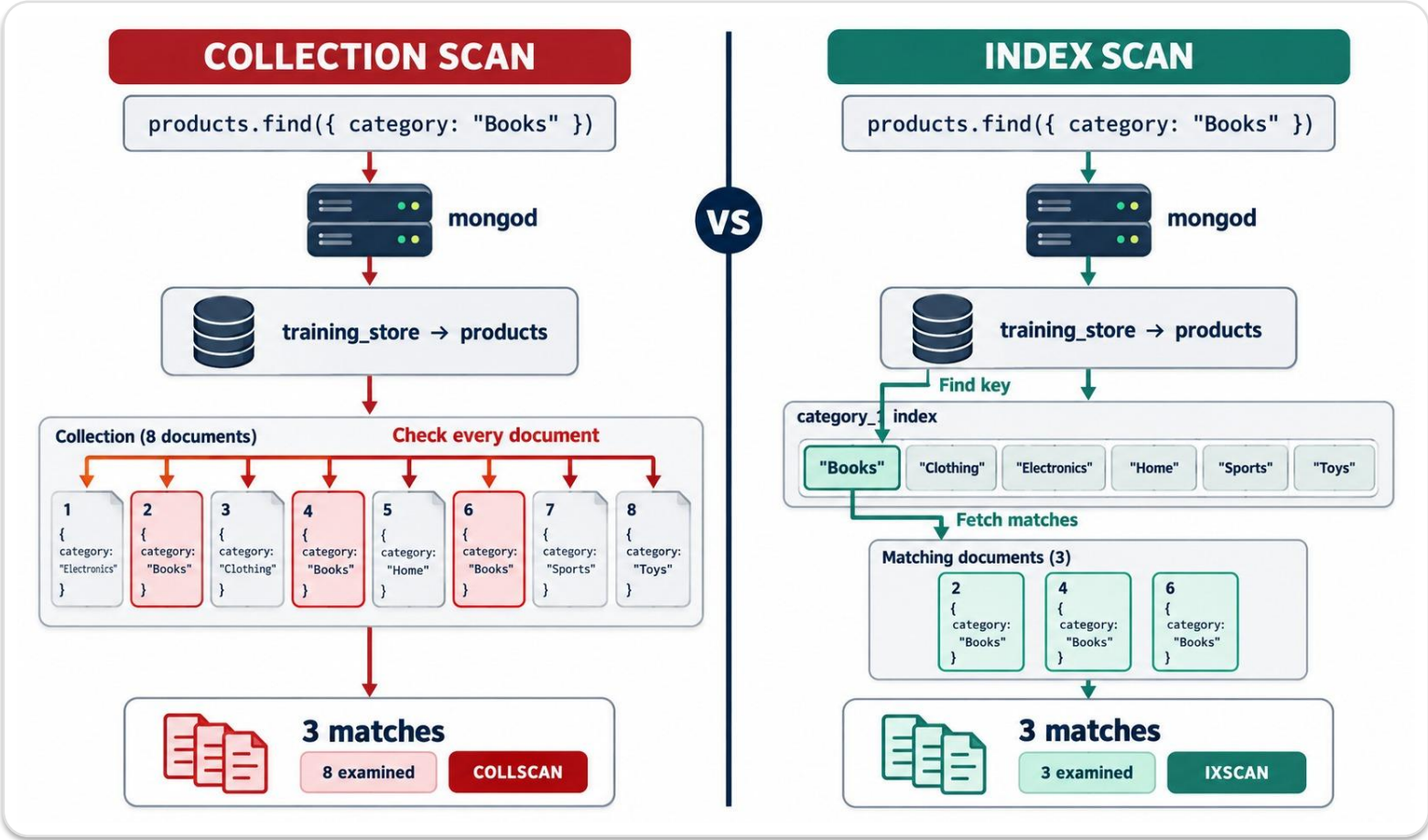
Slow queries queue up: users wait and the server works harder.

OUR DATASET 13 products and 17 orders scan instantly — watch counts, not the clock.

KEY TAKEAWAY An index changes how much work a query does — the result stays the same.

COLLECTION SCAN VS INDEX SCAN

Same query, same three matches — 8 documents examined versus 3.



COLLSCAN

Reads every document and tests the filter on each one.

IXSCAN

Finds the matching keys in the category index, then fetches only those documents.

The signal

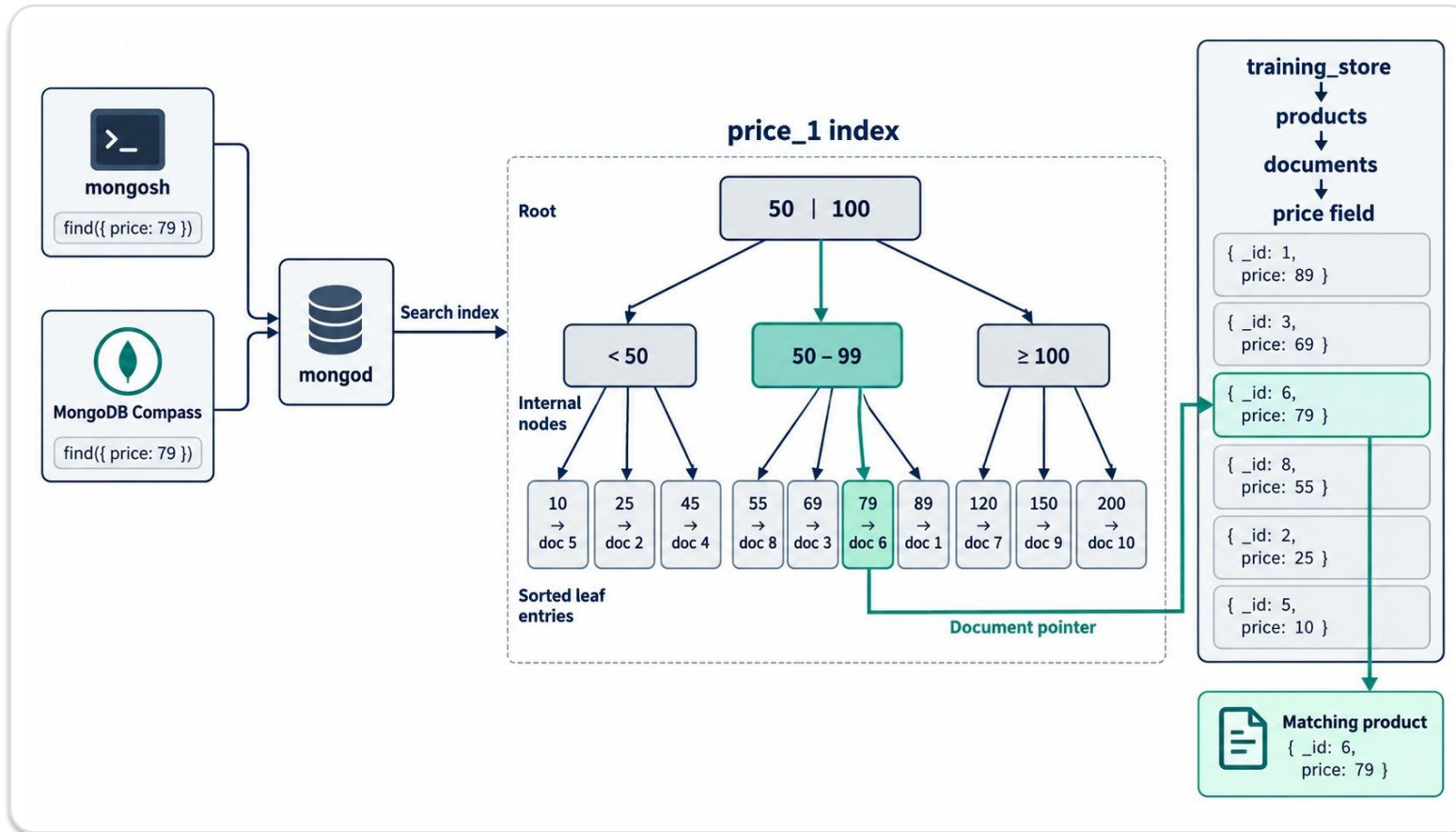
Examined versus returned: 8 : 3 against 3 : 3.

IN TRAINING_STORE category is "BOOK": 2 of 13 products.

KEY TAKEAWAY Compare documents examined with documents returned — the closer they are, the better the plan.

INSIDE AN INDEX: A SORTED TREE

An index is a B-tree of sorted keys, each pointing to a document.



Root and internal nodes

Split the key range: below 50, 50–99, 100 and above.

Sorted leaf entries

Each key points to its document: 79 → doc 6.

Why it's fast

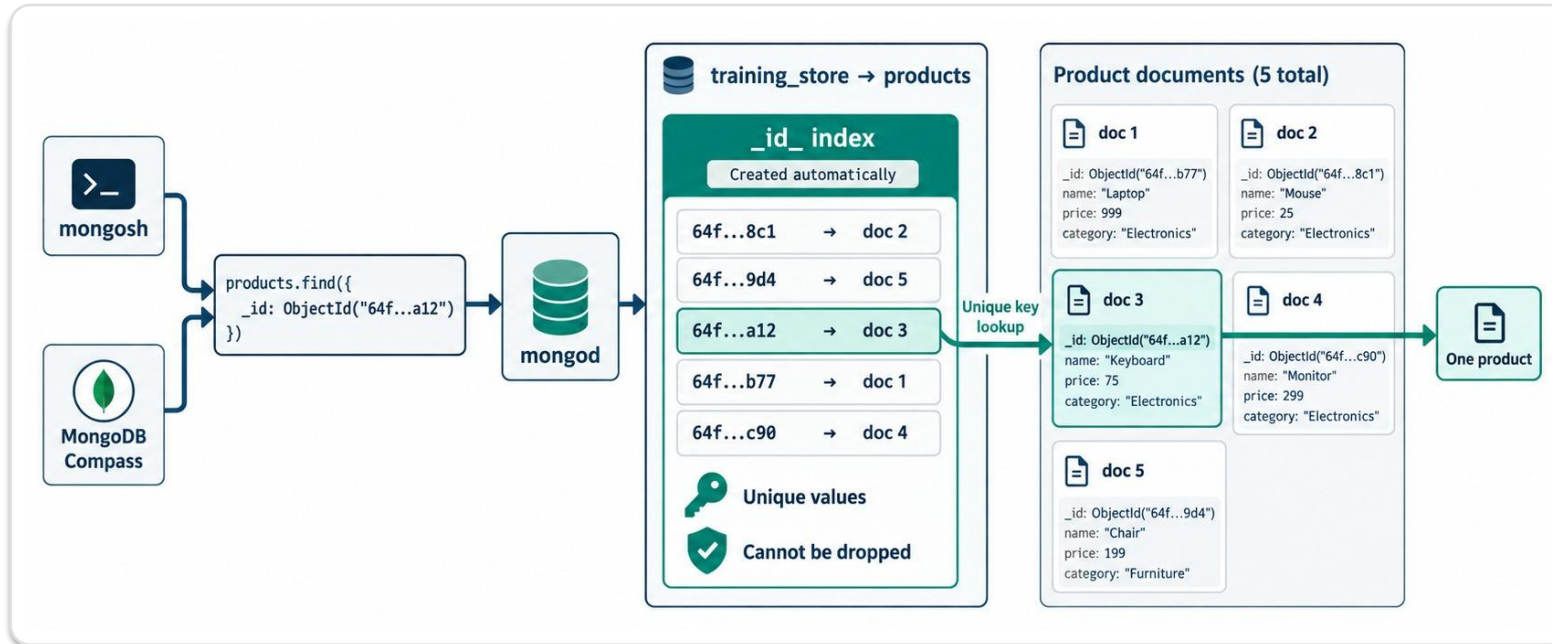
A few steps down the tree instead of reading every document.

SORTED The same order also serves ranges and sorts.

KEY TAKEAWAY An index keeps values in sorted order with pointers to documents — so lookups, ranges and sorts need no full scan.

THE DEFAULT _ID INDEX

Every collection gets a unique index on _id — automatically.



Created for you

Named `_id`; it exists before you create anything.

Unique

Two documents in one collection can't share an `_id`.

Can't be dropped

`dropIndexes()` removes every other index but keeps `_id`.

Always an index scan

```
db.products.find({
  _id: ObjectId("...")
})
```

KEY TAKEAWAY Lookups by `_id` are always indexed — every other query shape needs an index you design.

PART 2: MANAGE INDEXES

Create, list, hide and drop — by name.



SO FAR

- ✓ COLLSCAN reads every document; IXSCAN reads matching keys
- ✓ An index is a sorted tree of keys and pointers
- ✓ Every collection has a unique `_id` index

NEXT QUESTION

How do we create, inspect, hide and remove indexes safely?

IN THIS PART

- 1 What load.js already built
- 2 Creating indexes with clear names
- 3 Listing and dropping indexes
- 4 Hiding an index to test a removal

CREATE, LIST AND DROP NAMED INDEXES

On a fresh training_store — the output is what mongosh prints.

What load.js already built

```
db.products.getIndexes()  
  .map(i => i.name)  
  
[ '_id_', 'sku_unique' ]
```

Names

idx_<collection>_<fields> — easy to spot in explain() and \$indexStats.

✓ Create with a clear name

```
db.products.createIndex(  
  { category: 1, price: 1 },  
  { name:  
    "idx_products_category_price" }  
)  
  
idx_products_category_price
```

Several at once

createIndexes([...]) builds a list of index specifications in one command.

Drop by name — never _id_

```
db.products.dropIndex(  
  "idx_products_category_price"  
)  
  
{ nIndexesWas: 3, ok: 1 }
```

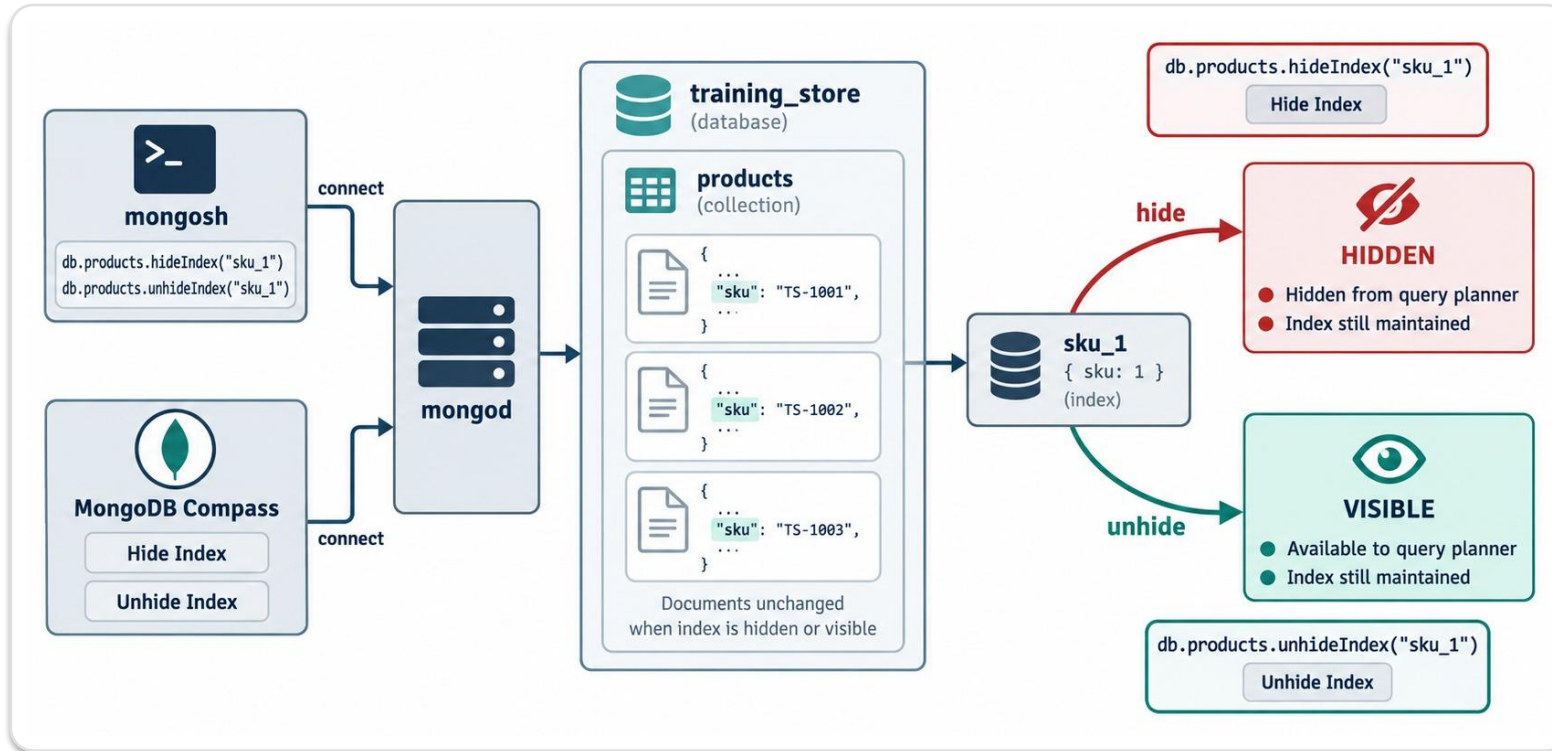
dropIndexes()

Removes all but _id_ — including unique rules like sku_unique.

KEY TAKEAWAY Name every index after its collection and fields — and list before you create or drop anything.

HIDE AN INDEX BEFORE YOU DROP IT

hideIndex() takes an index out of the planner's choices without deleting it.



Test a removal

```
db.products.hideIndex(
  "idx_products_category_price")
// run the queries + explain()
db.products.unhideIndex(
  "idx_products_category_price")
```

Hidden

The planner ignores it; every write still maintains it.

Visible again

`unhideIndex()` restores it at once — no rebuild.

UNIQUE STILL APPLIES A hidden unique index still blocks duplicates.

KEY TAKEAWAY Hide, measure, then drop — hiding is instant to undo, dropping means a rebuild.

PART 3: COMPOUND INDEXES AND ESR

Real queries filter and sort on several fields.



SO FAR

- ✓ Indexes are created, listed and dropped by name
- ✓ Hide first, measure, then drop
- ✓ load.js already enforces a unique sku

NEXT QUESTION

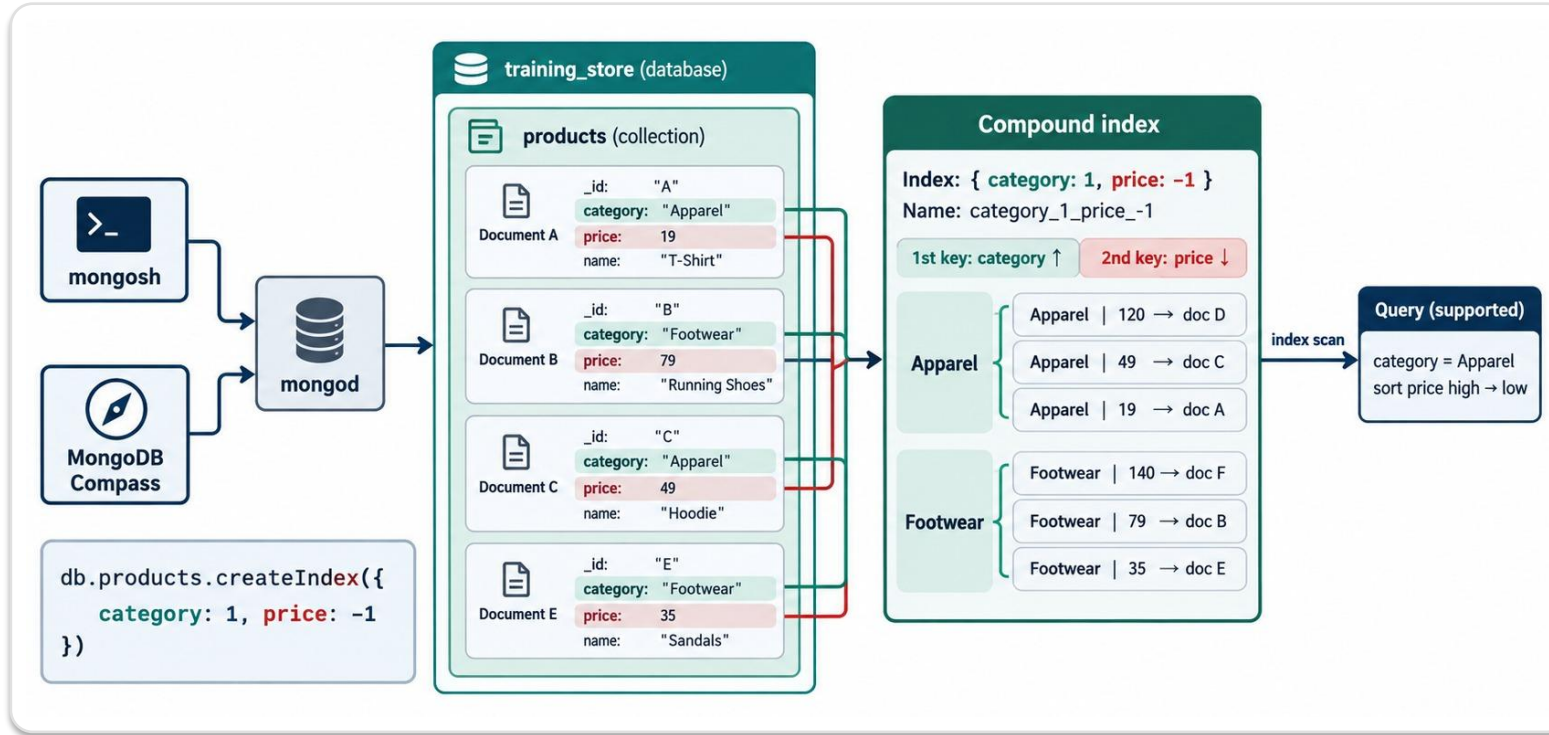
Which fields go into one index — and in what order?

IN THIS PART

- 1 Compound indexes
- 2 Why field order matters
- 3 The prefix rule
- 4 Equality, Sort, Range
- 5 Order history before and after
- 6 Sort direction and reverse scans
- 7 Exercise: the fulfillment queue

COMPOUND INDEXES

One index, several fields: sorted by category, then by price.



Sorted by the first key

All Apparel keys, then all Footwear keys.

Then by the second

Inside each category, price runs high to low because of price: -1.

Filter + sort

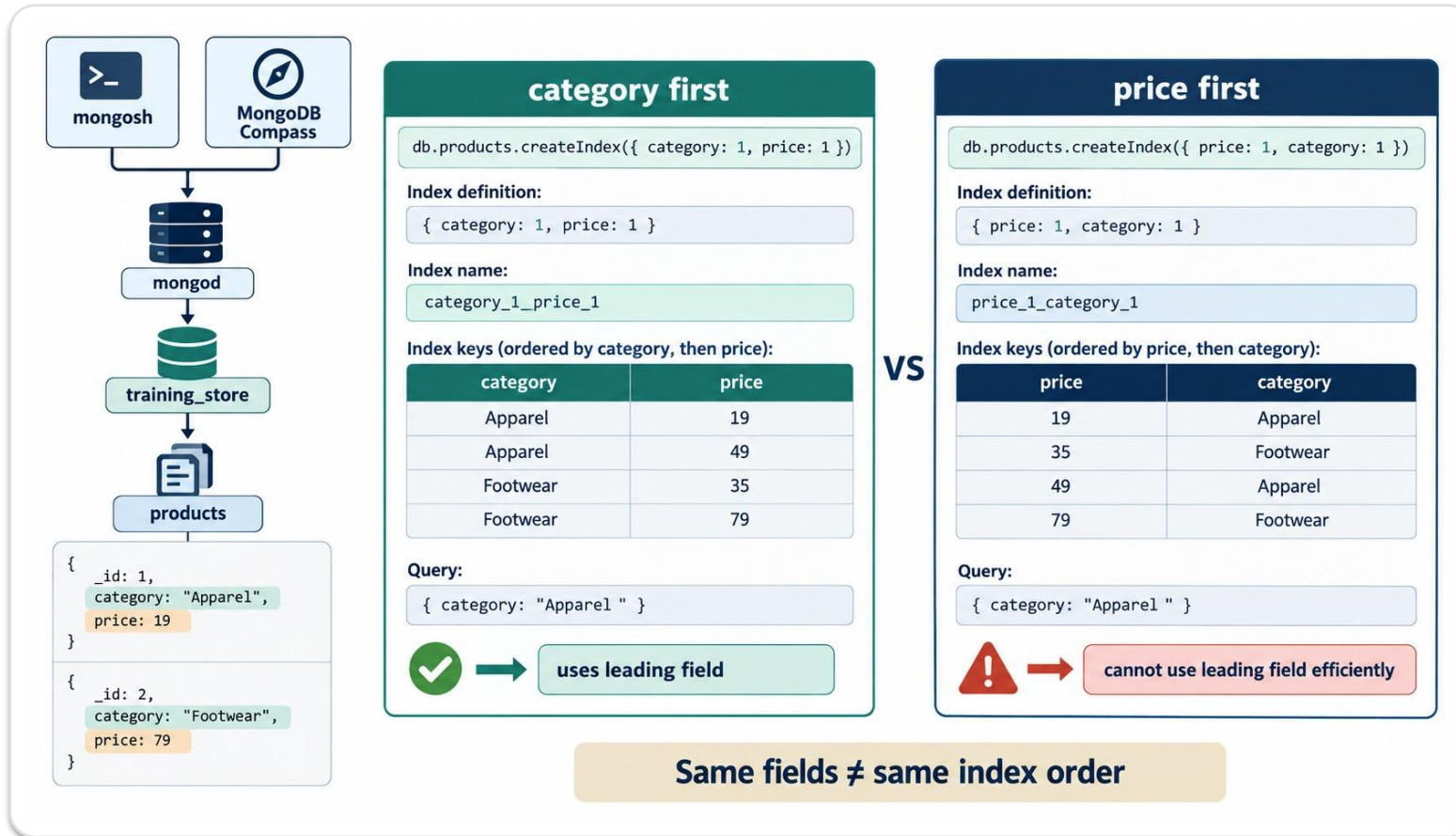
category = Apparel sorted by price is one run of keys.

DEFAULT NAME category_1_price_-1 — unless you set name.

KEY TAKEAWAY A compound index sorts by its first field, then the next — so one index can filter and sort at once.

FIELD ORDER MATTERS

Same two fields, two different indexes.



category first

Keys grouped by category — a category query reads one block.

price first

Categories are scattered through the price order.

Our catalog

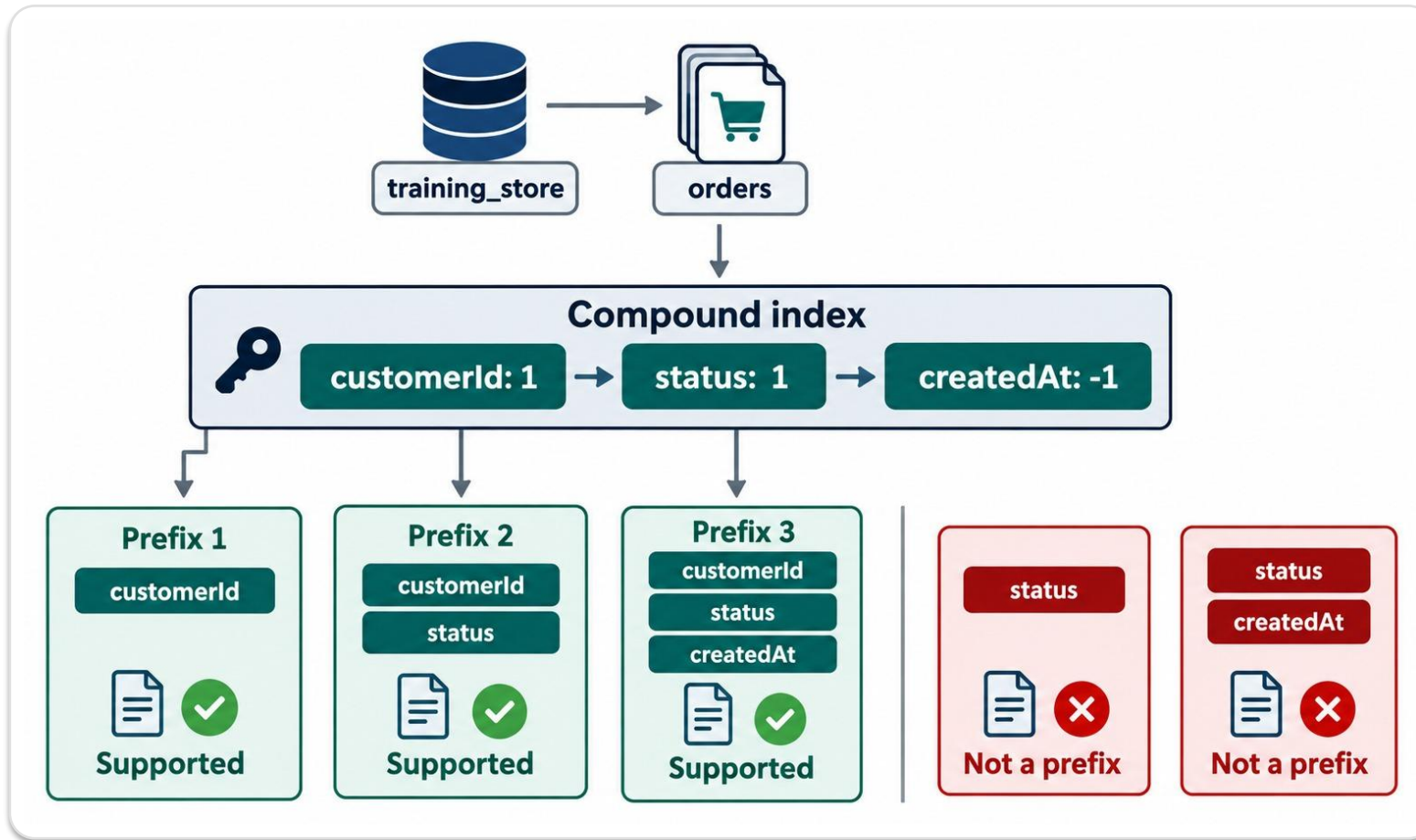
It always filters by category — so category leads.

SAME FIELDS ≠ SAME INDEX Choose the order from the query.

KEY TAKEAWAY `{ category: 1, price: 1 }` and `{ price: 1, category: 1 }` serve different queries — field order is a design decision.

THE PREFIX RULE

A compound index serves queries on its leading fields.



Supported prefixes

customerId · customerId + status · all three.

Not a prefix

status alone, or status + createdAt, skips the first field.

In training_store

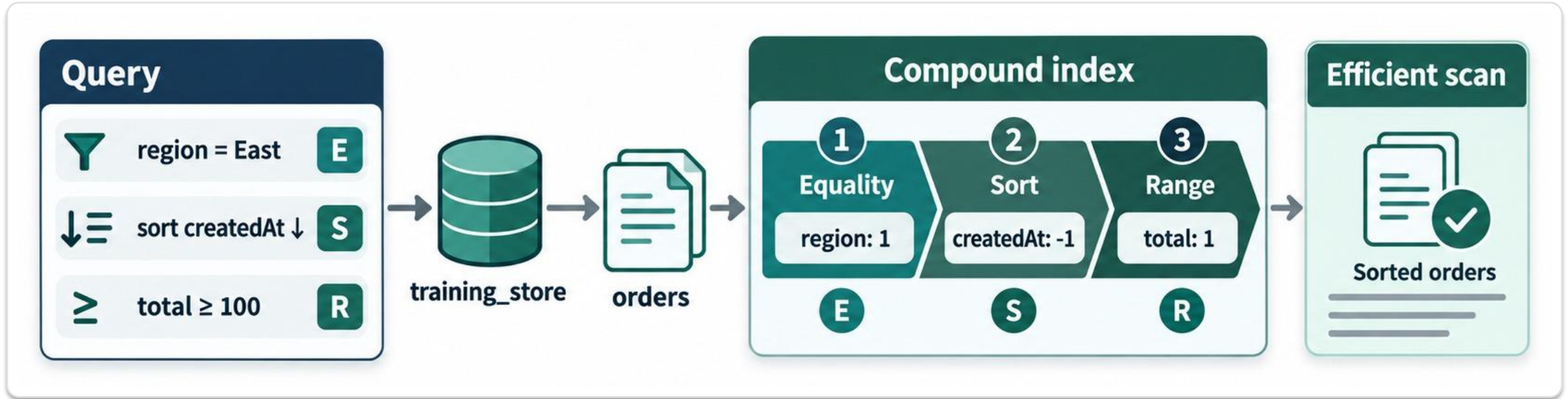
status stands for paymentStatus: { customerId, paymentStatus, createdAt }.

REDUNDANCY { customerId: 1 } adds little once this index exists.

KEY TAKEAWAY Queries can use a compound index through its leading fields — read the prefixes from the left.

EQUALITY, SORT, RANGE (ESR)

A starting order for compound-index fields.



E — Equality

Exact matches first: region = East.

S — Sort

Then the sort field in its direction: createdAt: -1.

R — Range

Range conditions last: total ≥ 100.

A GUIDELINE Confirm with `explain()` — data can change the order.

KEY TAKEAWAY Equality, then Sort, then Range: the index narrows first, delivers the order next, and scans the range last.

ORDER HISTORY: BEFORE AND AFTER THE INDEX

Lab 5, Steps 1–3: C101's orders, newest first (explain output trimmed).

✗ Before: no matching index

```
db.orders.find({ customerId: cid })  
  .sort({ createdAt: -1 })  
  .explain("executionStats")
```

```
winningPlan: SORT  
  inputStage: COLLSCAN  
nReturned:      5  
totalKeysExamined: 0  
totalDocsExamined: 17
```

cid

```
db.customers.findOne({ customerNumber: "C101"  
})._id — never a copied ObjectId.
```

Before

All 17 orders read, then sorted in memory by the SORT stage.

✓ After: idx_orders_customer_date

```
db.orders.createIndex(  
  { customerId: 1, createdAt: -1 },  
  { name: "idx_orders_customer_date" })
```

```
winningPlan: FETCH  
  inputStage: IXSCAN  
nReturned:      5  
totalKeysExamined: 5  
totalDocsExamined: 5
```

After

5 keys, 5 documents, 5 orders — the index order replaces SORT.

KEY TAKEAWAY Equality on customerId and a matching sort on createdAt turn 17 reads and a sort into 5 reads in order.

WHICH SORTS CAN ONE INDEX SERVE?

Index { customerId: 1, createdAt: -1 }, filtering one customer.

Sort requested	Index walk	SORT stage?
{ createdAt: -1 }	Forward	No
{ createdAt: 1 }	Backward	No
{ customerId: -1, createdAt: 1 }	Full reverse	No
{ customerId: 1, createdAt: 1 }	Mixed directions	Yes
{ createdAt: -1 }, no customer filter	Not in date order	Yes

Same or full reverse

An index can be walked forwards or backwards — never half of each.

Mixed directions

Need an index built in that mix, such as { customerId: 1, createdAt: 1 }.

ONE FIELD For a single-field index, direction doesn't matter.

KEY TAKEAWAY One compound index serves its own sort order and the exact reverse — mixed directions need their own index.

EXERCISE: INDEX THE FULFILLMENT QUEUE

Lab 5, Step 4: a query shape the history index can't serve.

PRACTICE EXERCISE

10 MIN

SCENARIO

Warehouse staff list PROCESSING orders, oldest first. `idx_orders_customer_date` already exists.

```
db.orders.find(
  { fulfillmentStatus:
    "PROCESSING" }
).sort({ createdAt: 1 })
```

YOUR TASKS

- 1 Label the equality and the sort field
- 2 Say why `idx_orders_customer_date` can't help
- 3 Write the `createIndex` command, with a name
- 4 Predict the results and the new plan

EXPECTED OUTCOME

- ✓ E: fulfillmentStatus · S: createdAt
- ✓ Its leading field is customerId
- ✓ { fulfillmentStatus: 1, createdAt: 1 }
- ✓ Named
idx_orders_fulfillment_date
- ✓ O6202, O6302, O6401 —
IXSCAN, no SORT

KEY TAKEAWAY Every important query shape needs an index that leads with its own equality fields.

PART 4: SPECIALIZED INDEX TYPES

Arrays, business rules, subsets and expiry.



SO FAR

- ✓ Compound indexes sort by the first field, then the next
- ✓ Queries use an index through its prefix
- ✓ ESR: equality, then sort, then range

NEXT QUESTION

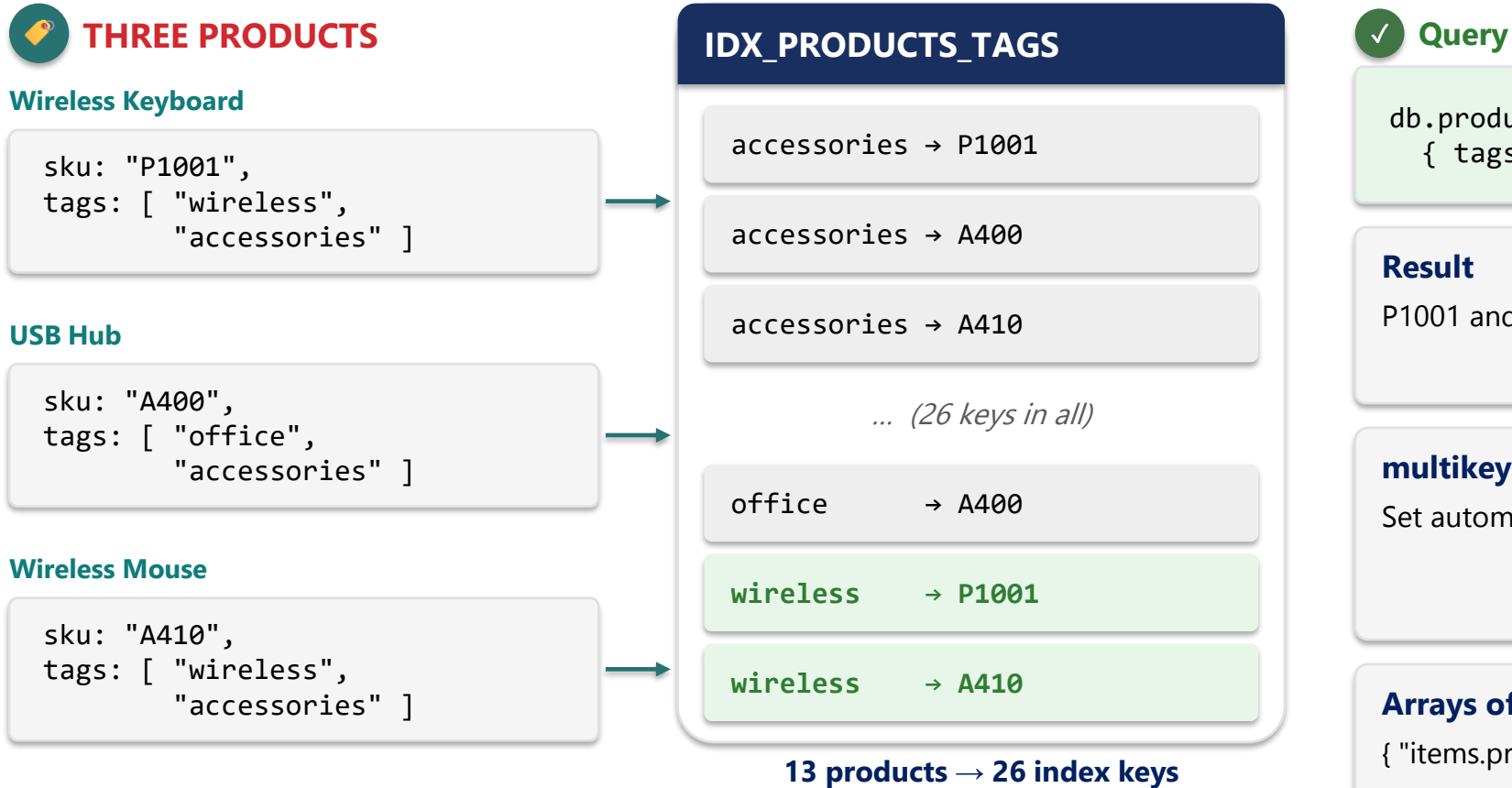
What if the field is an array, must be unique, or matters only for some documents?

IN THIS PART

- 1 Multikey indexes on arrays
- 2 Unique indexes
- 3 Partial indexes
- 4 Sparse indexes
- 5 TTL indexes
- 6 Text, wildcard, geospatial, hashed
- 7 Choosing the right type

MULTIKEY INDEXES ON ARRAYS

One index key for every array value — real tags from training_store products.



KEY TAKEAWAY Index an array field and MongoDB makes a multikey index — one key per value, so each array element is searchable.

UNIQUE INDEXES ENFORCE BUSINESS RULES

A duplicate SKU is rejected by the database — not just by the application.

Rebuild with the course name

```
// load.js built sku_unique
db.products.dropIndex(
  "sku_unique")
db.products.createIndex(
  { sku: 1 },
  { unique: true,
    name: "uq_products_sku" })
```

✖ A duplicate is rejected

```
db.products.insertOne({
  sku: "L100",
  name: "Duplicate SKU test"
})

MongoServerError: E11000
duplicate key error ...
index: uq_products_sku
dup key: { sku: "L100" }
```

Case-insensitive email

```
db.customers.createIndex(
  { "contact.email": 1 },
  { unique: true,
    name:
      "uq_customers_email_ci",
    collation: { locale: "en",
                  strength: 2 } })
```

One key, one index

Two indexes can't share { sku: 1 } — drop sku_unique first.

E11000

The insert fails and nothing is written.

Collation

strength: 2 ignores case; queries must use the same collation.

KEY TAKEAWAY A unique index turns a business rule — one SKU, one email — into something the database guarantees.

PARTIAL INDEXES: INDEX ONLY WHAT YOU QUERY

Lab 5, Step 10: only active products enter this index.

Create it

```
db.products.createIndex(  
  { category: 1, price: 1 },  
  {  
    name: "idx_active_products_category_price",  
    partialFilterExpression: { active: true }  
  }  
)  
  
// 12 of 13 products are indexed:  
// L190 is inactive
```

With and without active: true

```
// can use the partial index  
db.products.find({ category: "LAPTOP",  
  active: true,  
  price: { $lte: Decimal128("2000.00") } })  
// → L100, L110  
  
// can't: L190 would be missing  
db.products.find({ category: "LAPTOP",  
  price: { $lte: Decimal128("2000.00") } })
```

Smaller

Fewer keys to store and to update on every write.

The query must match

Only queries that include active: true can use it.

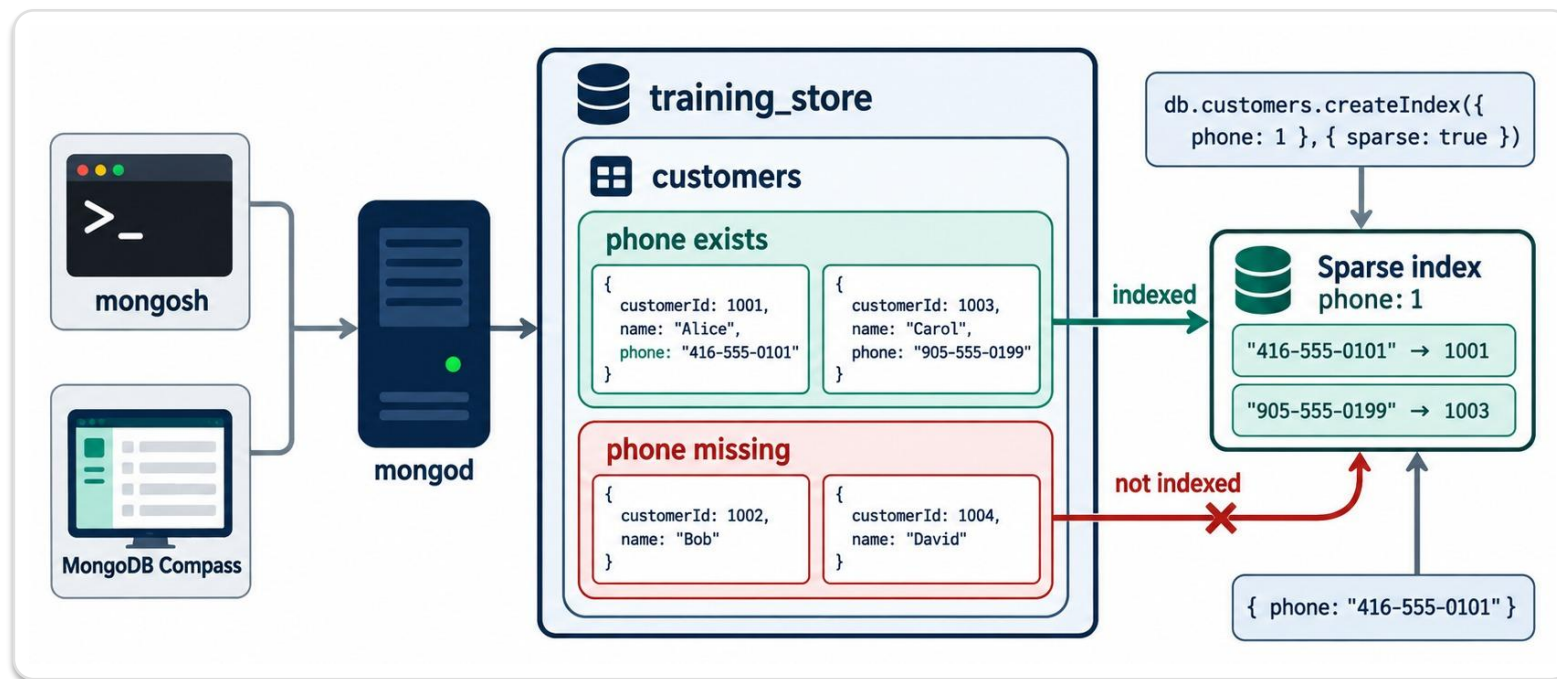
vs sparse

Partial takes any filter; sparse only checks that a field exists.

KEY TAKEAWAY A partial index covers only the documents the application asks for — and only queries that ask the same can use it.

SPARSE INDEXES

Only documents that have the field get an index entry.



Field present

Indexed — Alice and Carol have a phone.

Field missing

No entry — Bob and David are skipped.

training_store twist

C204's phone is null: it exists, so it's indexed. C515 has none.

PREFER PARTIAL A partial filter on \$exists says the same, explicitly.

KEY TAKEAWAY Sparse indexes skip documents without the field — partial indexes do the same job with a clearer rule.

TTL INDEXES REMOVE EXPIRED DOCUMENTS

Lab 5, Step 10: a dedicated sessions collection — never orders.

Sessions that expire

```
db.sessions.insertMany([
  { sessionId: "S1", user: "aisha",
    expiresAt: new Date(
      Date.now() + 60 * 60 * 1000) },
  { sessionId: "S-EXPIRED",
    user: "temp",
    expiresAt: new Date(
      Date.now() - 60 * 1000) }
])
```

✓ The TTL index

```
db.sessions.createIndex(
  { expiresAt: 1 },
  { expireAfterSeconds: 0,
    name: "ttl_sessions_expires_at" })

// about 60 s later the TTL monitor
// deletes S-EXPIRED; S1 stays
```

expireAfterSeconds: 0

Each document expires at its own expiresAt date.

Not instant

A background task runs about every 60 seconds.

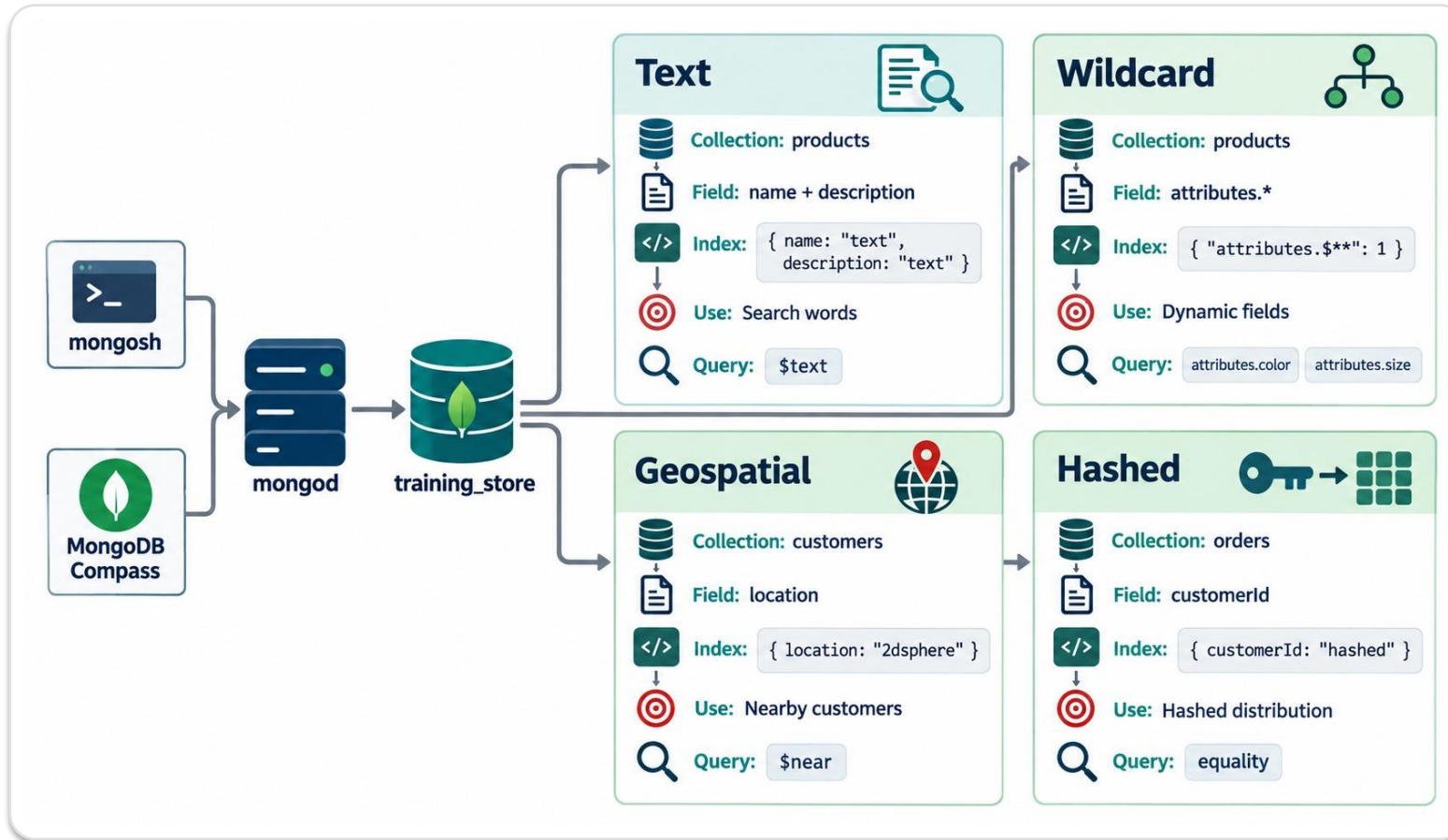
Never for records you keep

Orders, payments and audit data stay out of TTL.

KEY TAKEAWAY A TTL index deletes expired documents for you — use it only for data that is meant to disappear.

TEXT, WILDCARD, GEOSPATIAL AND HASHED

Four specialized types for four special query shapes.



Text

\$text word search — on name, since our products have no description.

Wildcard

{ "attributes.\$**": 1 } — our attributes differ by category.

Geospatial

2dsphere on GeoJSON points, longitude first; \$near.

Hashed

Equality only; spreads values for hashed sharding — Module 7.

KEY TAKEAWAY Specialized indexes serve special query shapes — word search, dynamic fields, location and shard distribution.

CHOOSING THE RIGHT INDEX TYPE

Start from what the query must do.

The query must...	Index type	training_store example
Match or sort one field	Single-field	{ sku: 1 }
Filter and sort on several	Compound	{ customerId: 1, createdAt: -1 }
Forbid duplicates	Unique	sku, customerNumber, orderNumber
Match array values	Multikey	{ tags: 1 }
Cover some documents only	Partial	active: true products
Skip a missing field	Sparse	contact.phone
Expire documents	TTL	sessions.expiresAt
Search words / dynamic paths	Text / Wildcard	name / attributes.\$**

Location and sharding

2dsphere for coordinates; hashed for hashed shard keys.

One index, many shapes

A compound index often serves several queries through its prefix.

RULE Pick the type from the query, not from the field.

KEY TAKEAWAY Name the query's job first — the index type follows from it.

PART 5: EXPLAIN() AND COVERED QUERIES

Stop guessing — read the plan.



SO FAR

- ✓ Arrays get multikey indexes automatically
- ✓ Unique, partial, sparse and TTL add rules and limits
- ✓ The query decides the index type

NEXT QUESTION

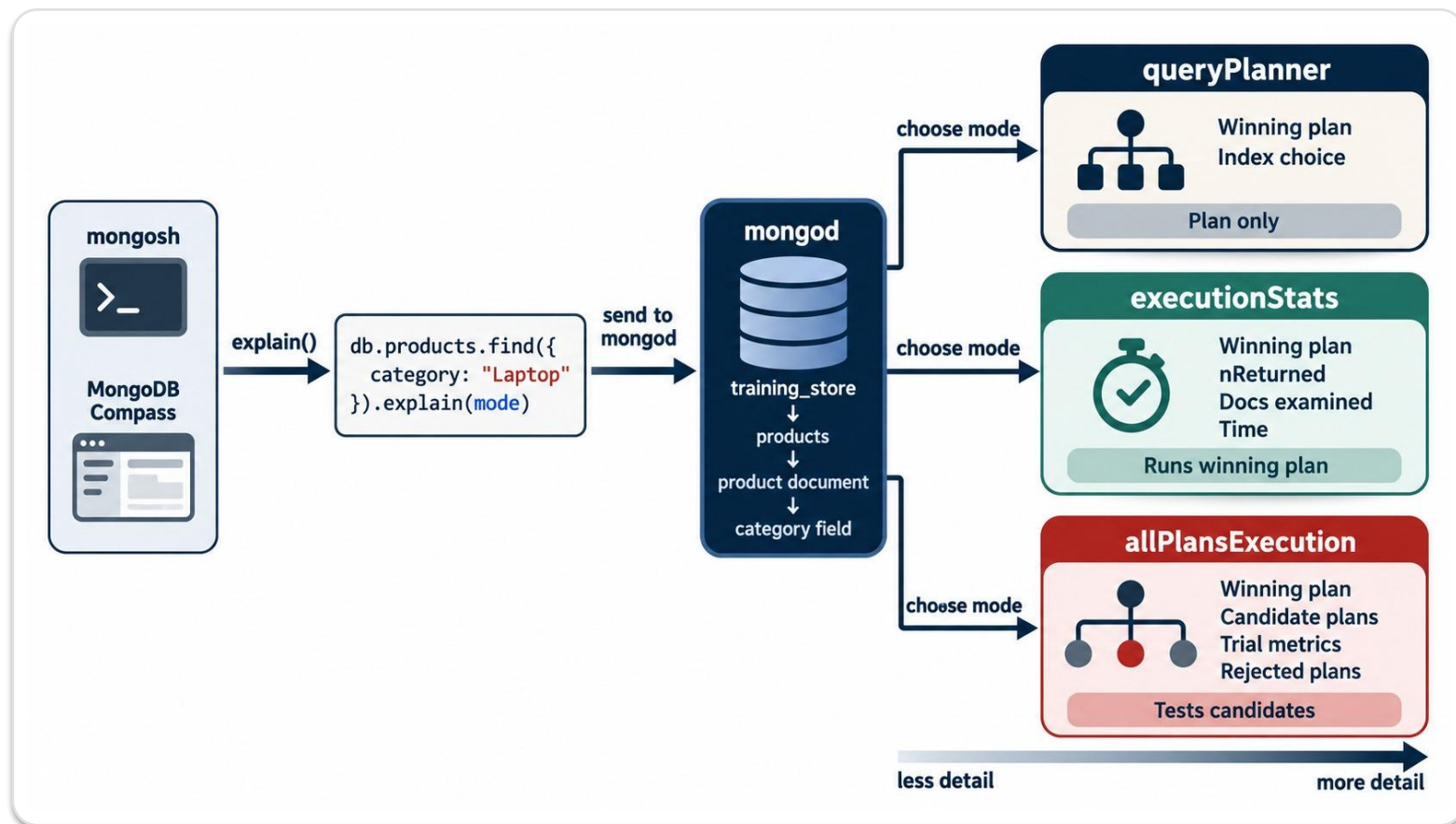
How do we prove an index works — and when can a query skip the documents entirely?

IN THIS PART

- 1 explain() verbosity modes
- 2 Reading executionStats
- 3 Spotting a blocking sort
- 4 Covered queries
- 5 Proving coverage in mongosh
- 6 Exercise: diagnose three plans

EXPLAIN(): THREE LEVELS OF DETAIL

queryPlanner, executionStats and allPlansExecution.



queryPlanner

The winning plan only — the query isn't run.

executionStats

Runs the winning plan: returned, keys and documents examined, time.

allPlansExecution

Adds the candidate plans the planner tried and rejected.

FOR TUNING `explain("executionStats")` — as in every lab.

KEY TAKEAWAY `explain()` shows the plan; `executionStats` shows what that plan actually cost.

READING EXECUTIONSTATS

Lab 5, Step 7: active accessories by price, with idx_products_category_active_price.

✓ The query and its plan (trimmed)

```
db.products.find(
  { category: "ACCESSORY", active: true }
).sort({ price: 1 }).explain("executionStats")

queryPlanner.winningPlan:
  stage: "FETCH"
  inputStage:
    stage: "IXSCAN"
    indexName: "idx_products_category_active_price"
executionStats:
  nReturned: 5
  totalKeysExamined: 5
  totalDocsExamined: 5
```

winningPlan

FETCH over IXSCAN: keys first, then the matching documents. No SORT.

Keys : docs : returned

5 : 5 : 5 — nothing wasted.

Before the index

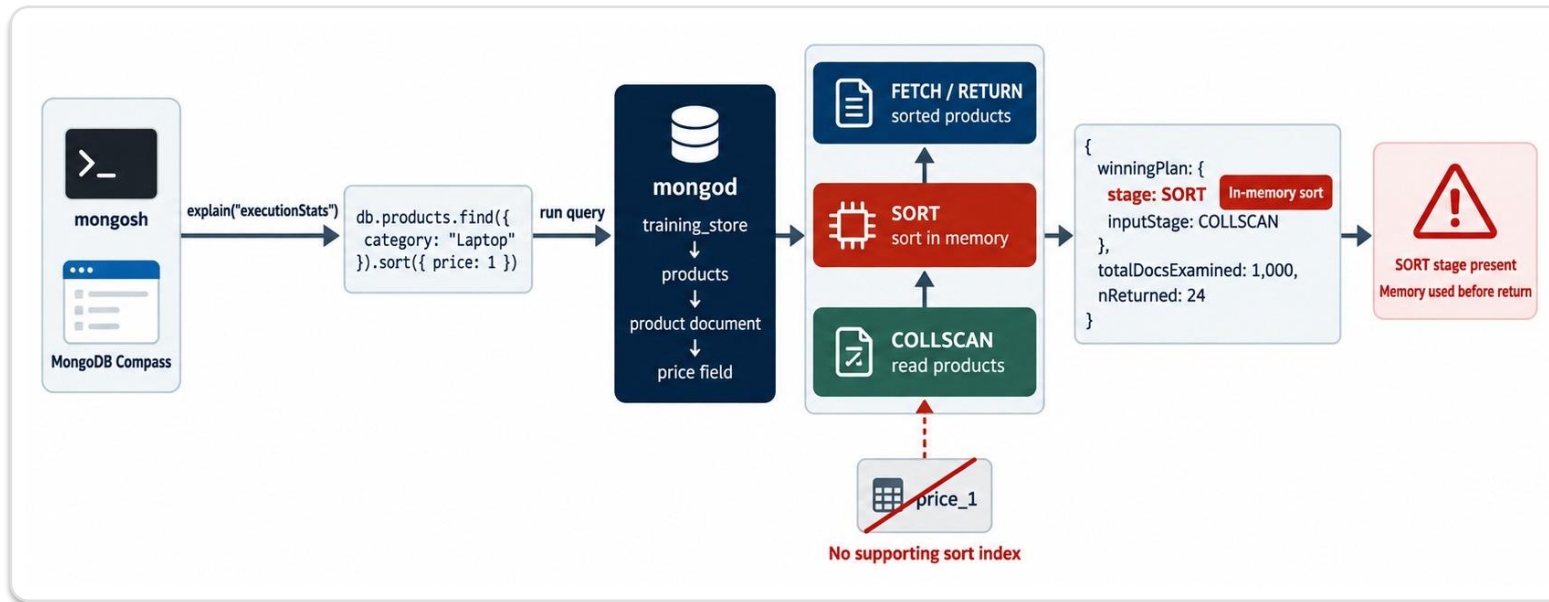
SORT over COLLSCAN: 0 keys, 13 documents, 5 returned.

XBAD Its price is the string "49.99" — it matches, and sorts after the numbers.

KEY TAKEAWAY Read the stage path first, then compare keys and documents examined with documents returned.

SPOTTING A BLOCKING SORT

A SORT stage means documents were sorted in memory.



The signal

stage: SORT above COLLSCAN or IXSCAN in winningPlan.

Why it hurts

Every match is held and sorted before the first result returns.

The fix

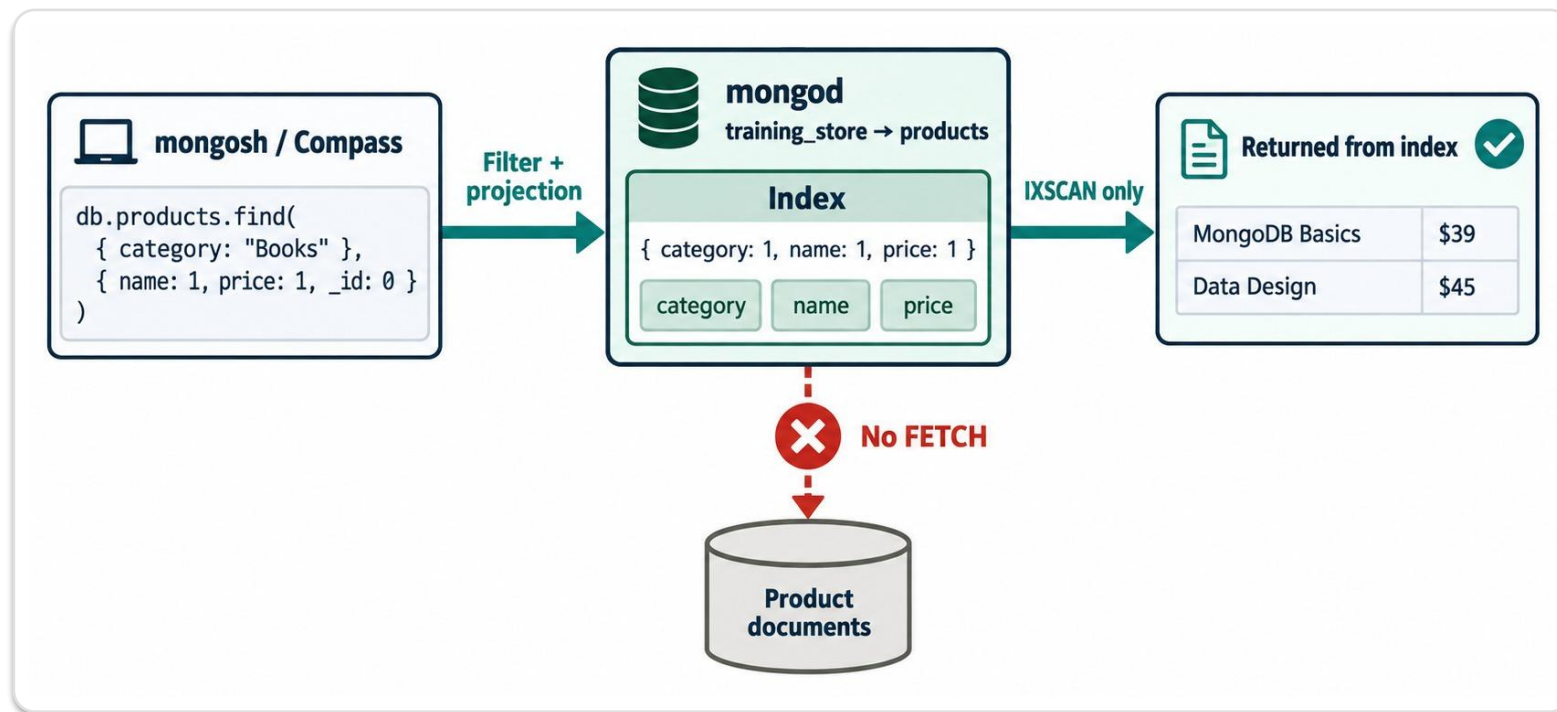
An index whose order matches the sort — after the equality fields.

MEMORY Large sorts use up to 100 MB, then spill to disk.

KEY TAKEAWAY A SORT stage is a missing or mis-ordered index — fix the index, not the query.

COVERED QUERIES

When the index holds every field the query needs, no document is read.



Filter in the index

category is the first key.

Output in the index

name and price are keys too.

_id excluded

_id isn't in the index, so the projection sets _id: 0.

THE SIGNAL IXSCAN with no FETCH;
totalDocsExamined: 0.

KEY TAKEAWAY A covered query answers from the index alone — every filtered and returned field must be in it.

PROVING COVERAGE IN MONGOSH

Lab 5, Step 11: idx_products_category_price_name on the two books (trimmed).

✓ Covered

```
db.products.createIndex(  
  { category: 1, price: 1, name: 1 },  
  { name: "idx_products_category_price_name" })  
  
db.products.find({ category: "BOOK" },  
  { _id: 0, name: 1, price: 1 })  
  
stage: PROJECTION_COVERED  
  inputStage: IXSCAN  
totalDocsExamined: 0
```

✗ Not covered

```
// _id comes back by default  
db.products.find({ category: "BOOK" },  
  { name: 1, price: 1 })  
  
// tags isn't in the index  
db.products.find({ category: "BOOK" },  
  { _id: 0, name: 1, tags: 1 })  
  
stage: PROJECTION_SIMPLE  
  inputStage: FETCH  
totalDocsExamined: 2
```

Three conditions

Filter indexed · output indexed · `_id` excluded or indexed.

The result

Query Cookbook 39.99, then MongoDB Fundamentals 59.99.

Arrays

A multikey field such as `tags` can't be covered.

KEY TAKEAWAY Confirm coverage with `explain()` — IXSCAN with no FETCH and zero documents examined.

EXERCISE: DIAGNOSE THREE PLANS

Read the numbers, name the problem, propose the fix.

PRACTICE EXERCISE

10 MIN

SCENARIO

Three explain("executionStats") summaries on a fresh training_store. A: name "USB Hub". B: PROCESSING orders by date. C: tags "wireless".

```
A  COLLSCAN
   keys 0 · docs 13 · returned 1
B  SORT ← COLLSCAN
   keys 0 · docs 17 · returned 3
C  FETCH ← IXSCAN idx_products_tags
   keys 2 · docs 2 · returned 2
```

YOUR TASKS

- 1 Mark each plan targeted or wasteful
- 2 Propose a fix for A
- 3 Propose a fix for B
- 4 Say why C needs no change

EXPECTED OUTCOME

- ✓ A and B wasteful · C targeted
- ✓ A: index name, or query by sku
- ✓ B: idx_orders_fulfillment_date
- ✓ B's SORT disappears with it
- ✓ C: 2 : 2 : 2 already

KEY TAKEAWAY Read stage, keys, documents and returned together — any one alone can mislead.

PART 6: SELECTIVITY, COSTS AND COMMON MISTAKES

Every index is a trade-off.



SO FAR

- ✓ explain("executionStats") proves what a query did
- ✓ A SORT stage means an in-memory sort
- ✓ Covered queries skip the documents entirely

NEXT QUESTION

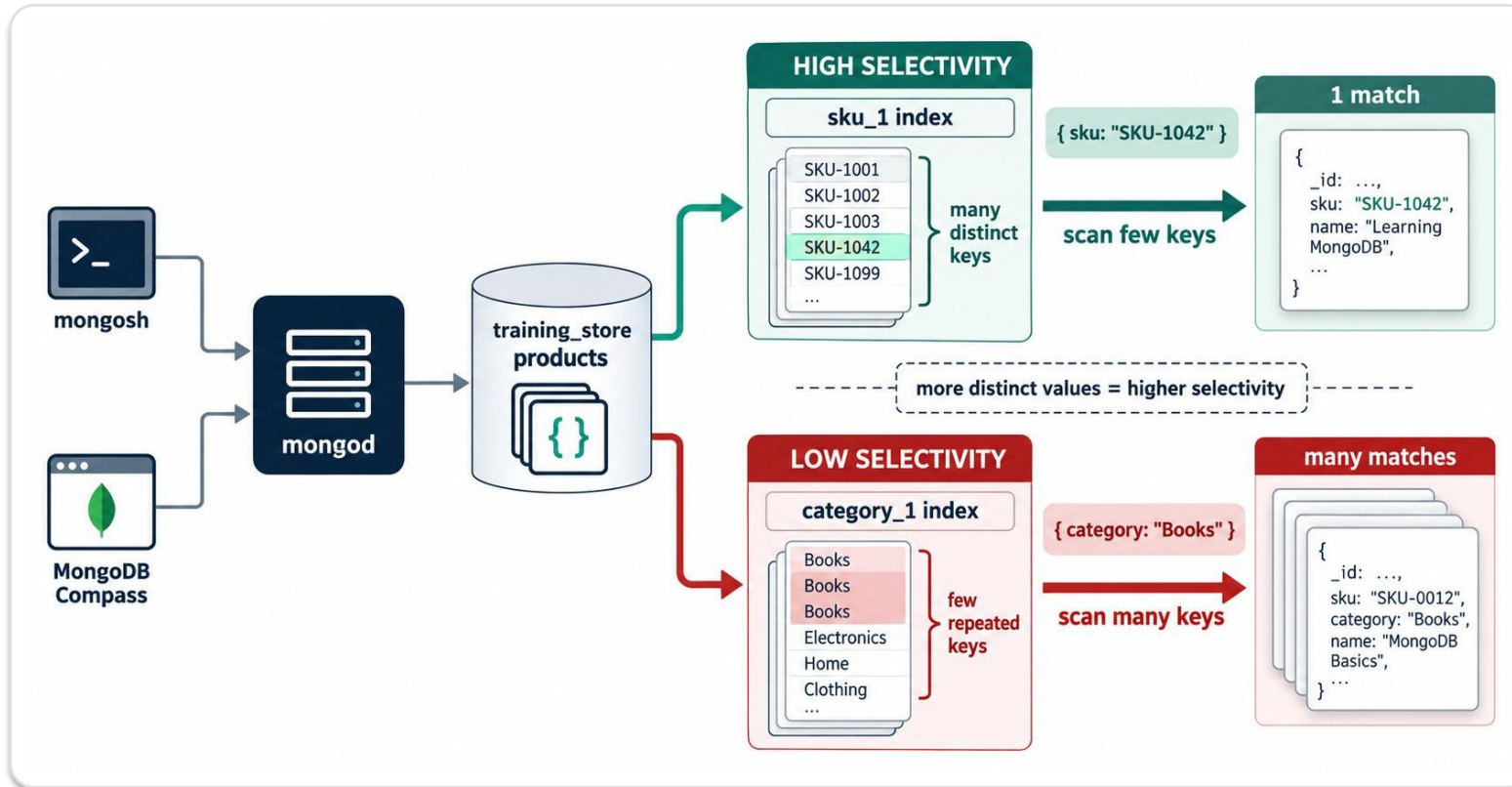
Is every index worth what it costs — and how do we keep the set healthy?

IN THIS PART

- 1 Index selectivity
- 2 Write, storage and memory costs
- 3 Usage statistics with \$indexStats
- 4 Changing indexes safely in production
- 5 Common indexing mistakes
- 6 An indexing strategy for training_store

INDEX SELECTIVITY

The more distinct values, the fewer keys a lookup scans.



High

sku: 13 distinct values in 13 products — one key per lookup.

Low

active: 12 of 13 products are true — barely narrows the search.

Combine

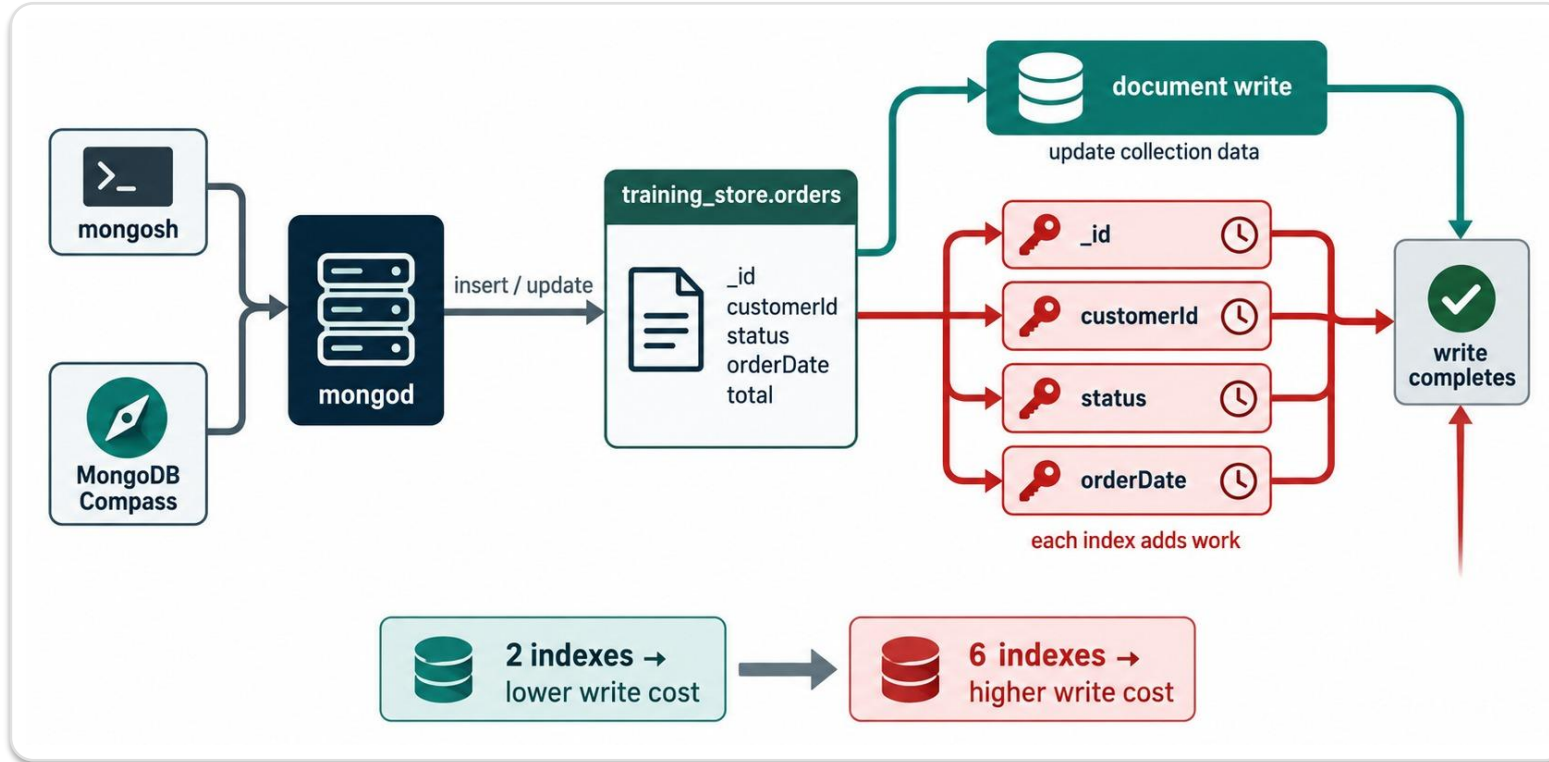
Put low-selectivity fields in a compound or partial index.

MEASURE Selectivity comes from your data — check it with `explain()`.

KEY TAKEAWAY An index narrows a search only as much as its values do — lead with fields that really narrow it.

EVERY INDEX COSTS WRITES

Each insert or update also updates the indexes it touches.



Write path

One order insert updates the document, **_id**, **orderId_unique** and every secondary index.

Storage and memory

Indexes take disk space and compete for RAM with the data.

Updates

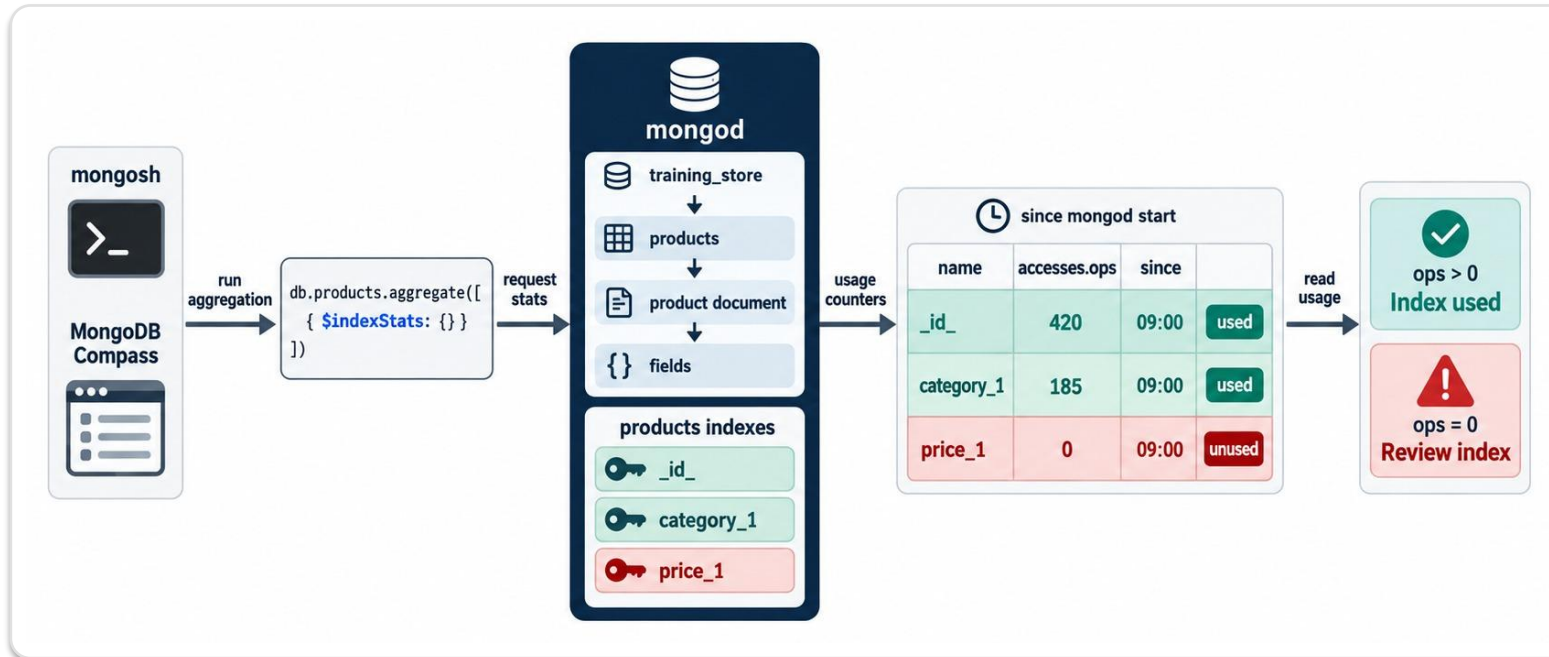
Changing an indexed field rewrites its index keys.

BALANCE Keep indexes that serve real, frequent queries.

KEY TAKEAWAY Every index speeds some reads and slows every write — keep the ones that earn their cost.

INDEX USAGE STATISTICS

\$indexStats counts how often each index was used.



Usage per index

```
db.products.aggregate([
  { $indexStats: {} }
]).map(s => ({ name: s.name,
  ops: s.accesses.ops })))
```

ops > 0

The planner is using the index.

ops = 0

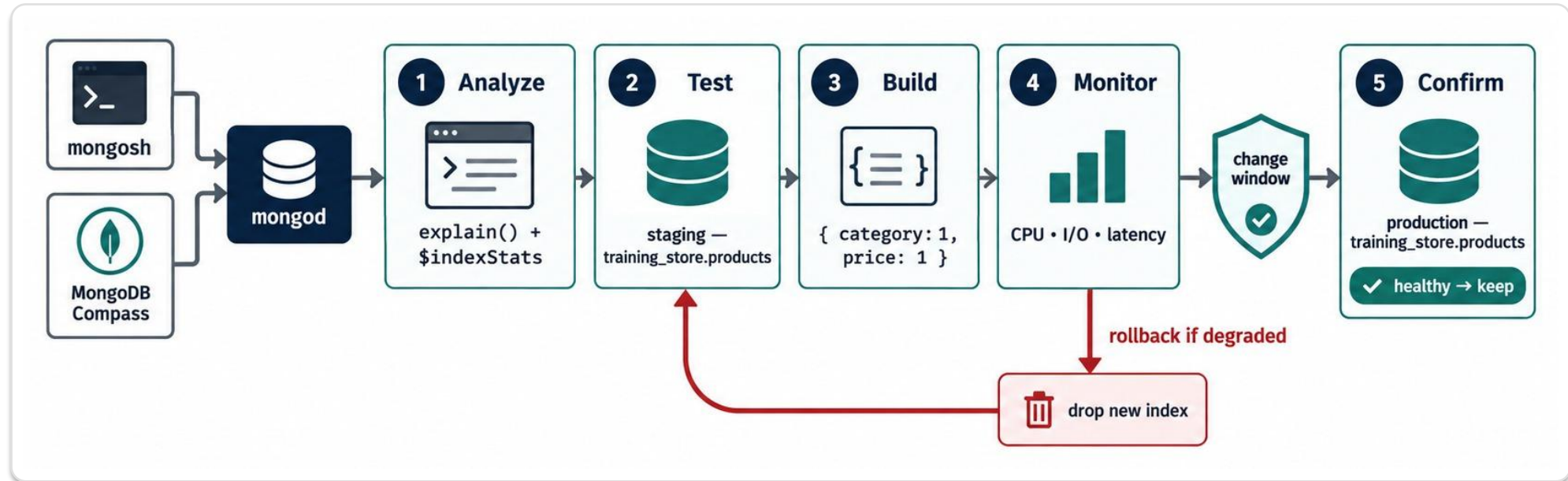
Review it: hide, measure, then decide.

SINCE RESTART Counters reset when mongod restarts.

KEY TAKEAWAY \$indexStats shows which indexes earn their keep — zero uses starts an investigation, not a drop.

CHANGING INDEXES SAFELY IN PRODUCTION

Analyze, test, build, monitor, confirm — with a rollback ready.



Analyze and test

explain() and \$indexStats; try it on staging data first.

Build and monitor

Build in a change window; watch CPU, I/O and latency.

Rollback

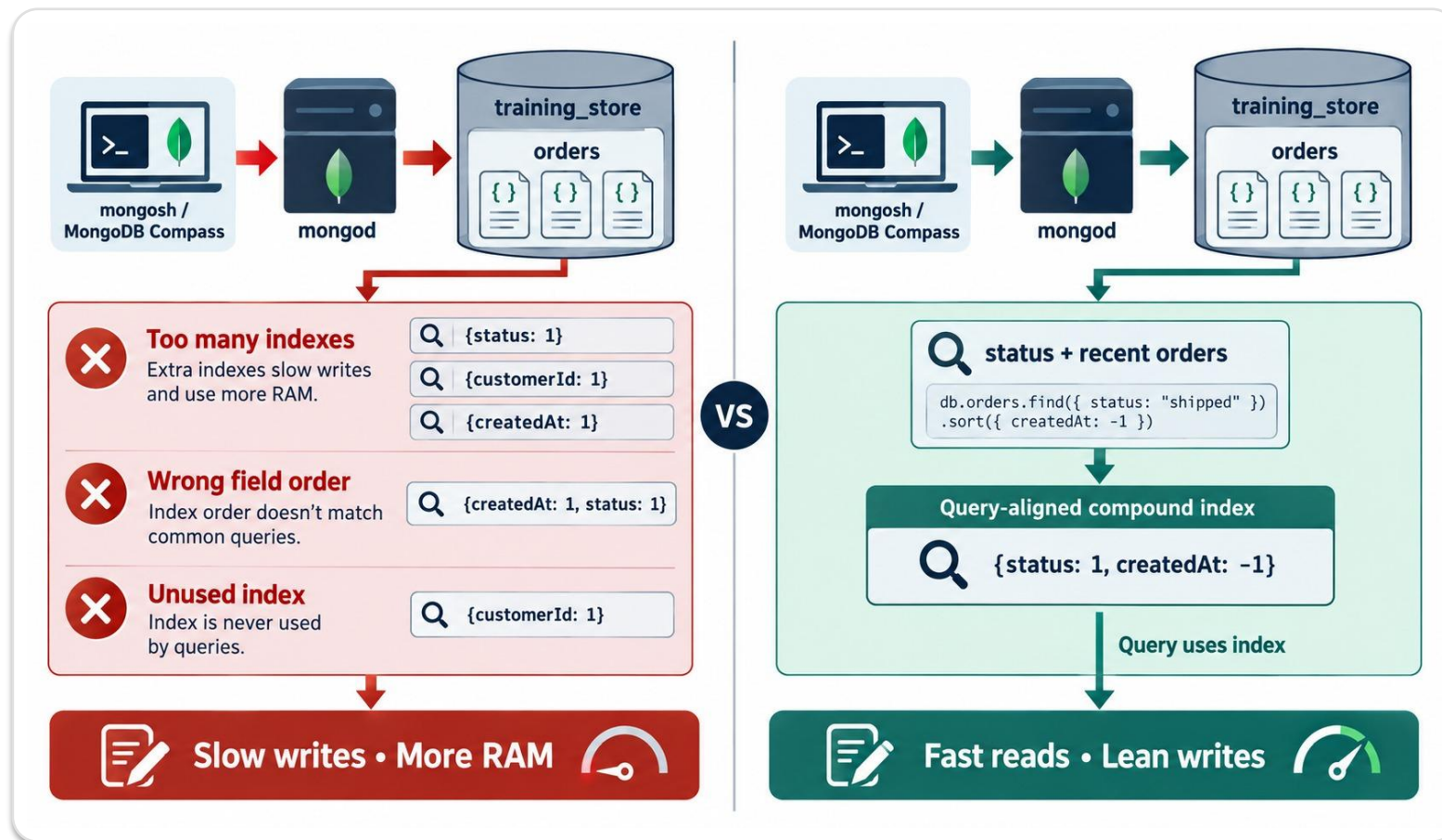
If it degrades, drop the new index — keep the createIndex command.

REMOVING Hide first; drop only when the queries stay healthy.

KEY TAKEAWAY Treat every index change as a production change: measured, monitored and reversible.

COMMON INDEXING MISTAKES

Too many, wrong order, never used — and the query-aligned fix.



Too many indexes

Every extra index slows writes and uses RAM.

Wrong field order

`{ createdAt, status }` can't serve `status + newest` first.

Unused indexes

`$indexStats ops = 0` — review them.

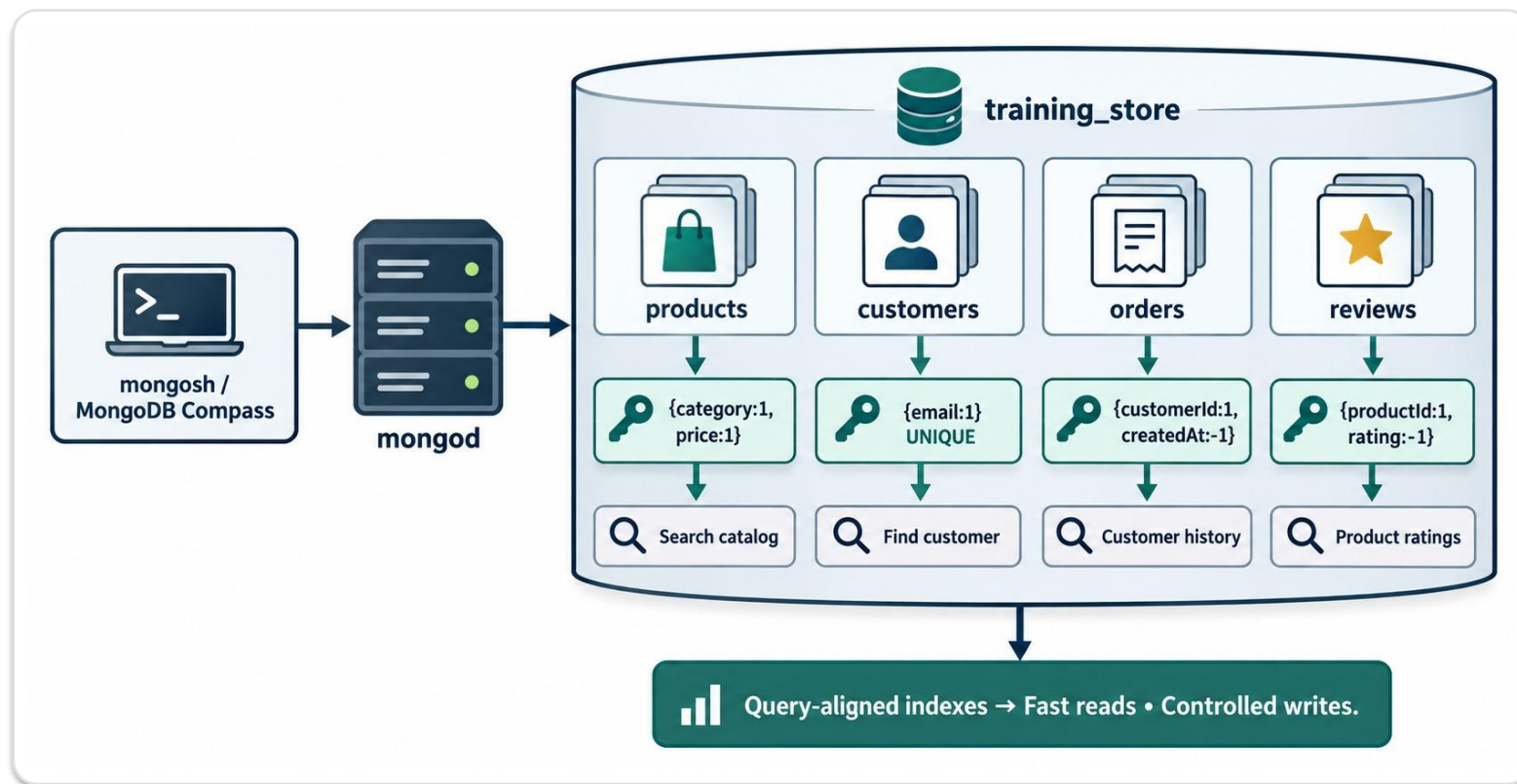
Also avoid

`hint()` as a permanent fix · timing tiny data · trusting any `IXSCAN`.

KEY TAKEAWAY Index the real query shapes, in ESR order — and review what nothing uses.

AN INDEXING STRATEGY FOR TRAINING_STORE

One index per important query shape — plus the integrity rules.



products

`{ category: 1, price: 1 }` for the catalog; unique sku.

customers

Unique `contact.email` — the diagram shortens it to `email`.

orders

`{ customerId: 1, createdAt: -1 }` for order history.

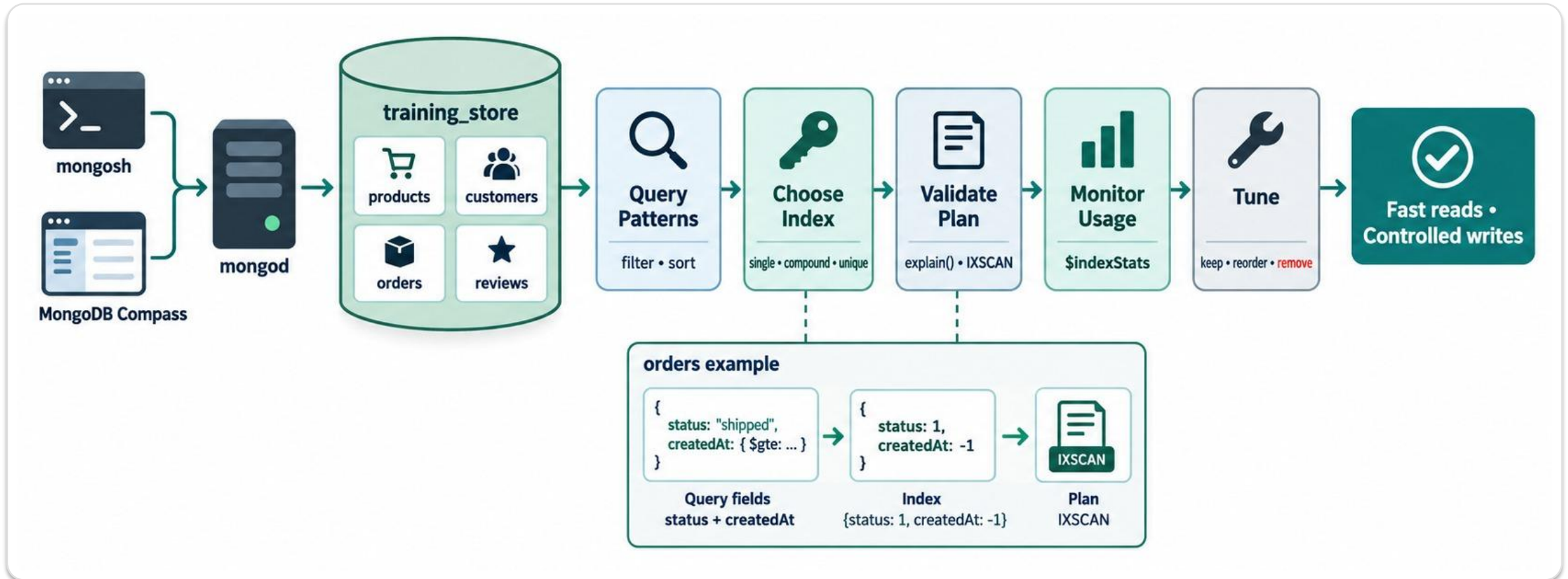
reviews

`{ productId: 1, rating: -1 }` for top ratings — or `createdAt: -1` for newest.

KEY TAKEAWAY A small set of query-aligned indexes gives fast reads with controlled writes.

MODULE 6 AT A GLANCE

Query patterns → choose index → validate plan → monitor usage → tune.



KEY TAKEAWAY Start from the query, prove the index with explain(), and keep reviewing what the index set costs.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What does a COLLSCAN stage tell you?

2 Which index does every collection have?

3 Why does field order matter in a compound index?

4 What does ESR stand for, and what is it for?

5 What makes an index on tags multikey?

6 How is a partial index different from a sparse one?

7 Why should a TTL index never go on orders?

8 Which three explain() numbers do you compare?

9 What three conditions make a query covered?

10 Why hide an index before dropping it?

EXERCISE 6.1 — APPLY EQUALITY–SORT–RANGE

Module 6 checkpoint A: three query shapes, three compound indexes.

OFFICIAL CHECKPOINT A

15 MIN

labs/day-03/exercises/exercise-6.1-apply-equality-sort-range.md

SCENARIO

Label equality, sort and range in each shape, then propose one compound index per shape.

- A products: ACCESSORY, active, price ≤ 100 , sort by price
- B orders: one customerId, createdAt in a date range, sort createdAt descending
- C orders: paymentStatus PAID, createdAt $\geq$ start, sort desc

YOUR TASKS

- 1 Label E, S and R in shape A
- 2 Label shape B — spot the shared field
- 3 Propose one index per shape
- 4 Say why paymentStatus leads in C

SOLUTION (AFTER DEBRIEF)

- ✓ A { category: 1, active: 1, price: 1 }
- ✓ B { customerId: 1, createdAt: -1 }
- ✓ C { paymentStatus: 1, createdAt: -1 }
- ✓ Equality first partitions the keys
- ✓ No index leads with a range

KEY TAKEAWAY ESR gives a strong first design — equality narrows, sort orders, range scans last.

EXERCISE 6.2 — IDENTIFY COVERED QUERIES

Module 6 checkpoint B: which projections can the index answer alone?

OFFICIAL CHECKPOINT B

15 MIN

labs/day-03/exercises/exercise-6.2-identify-covered-queries.md

SCENARIO

The index { category: 1, price: 1, name: 1 } exists. Decide which queries can be covered.

```
Q1 find({ category: "BOOK" },
      { _id: 0, name: 1, price: 1 })
Q2 find({ category: "BOOK" },
      { _id: 0, name: 1, price: 1,
        tags: 1 })
Q3 find({ category: "BOOK" },
      { name: 1 })
```

YOUR TASKS

- 1 Decide coverage for Q1
- 2 Decide coverage for Q2 and Q3
- 3 Write the three coverage conditions
- 4 Name the explain() signal to look for

SOLUTION (AFTER DEBRIEF)

- ✓ Q1 may be covered
- ✓ Q2 not — tags needs a FETCH
- ✓ Q3 not — _id returns by default
- ✓ Filter, output, _id handled
- ✓ IXSCAN with no FETCH

KEY TAKEAWAY A covered query needs every filtered and returned field in the index — including the decision about _id.

EXERCISE 6.3 — E-COMMERCE INDEX REVIEW

Module 6 checkpoint C: an index recommendation for ten workloads.

OFFICIAL CHECKPOINT C

30–45 MIN

labs/day-03/exercises/exercise-6.3-e-commerce-index-review.md

SCENARIO

Recommend indexes for ten recurring training_store workloads.

- | | |
|-------------------|------------------|
| 1 SKU lookup | 6 order history |
| 2 category browse | 7 paid reporting |
| 3 tag search | 8 newest reviews |
| 4 email lookup | 9 session expiry |
| 5 order number | 10 dynamic attrs |

YOUR TASKS

- 1 Tabulate shape and index for all ten
- 2 Mark unique, multikey, partial, TTL, wildcard
- 3 Give ESR roles and a cost note per compound
- 4 Flag an overlap; plan validation and rollback

SOLUTION (AFTER DEBRIEF)

- ✓ A ten-row table
- ✓ uq_products_sku, idx_products_tags
- ✓ TTL on sessions · wildcard attributes
- ✓ Overlap: category vs category + active + price
- ✓ Baseline → create → explain → rollback

KEY TAKEAWAY A good index plan names the query, the index, its cost and how to prove it — and how to undo it.

LAB 5 — INDEX AND EXPLAIN

Day 3 official lab: measure, index and measure again on a fresh training_store.

OFFICIAL LAB

40–50 MIN

labs/day-03/lab5/LAB-5-GUIDE.md



SIX CORE STEPS ON A FRESH LOAD

1

Baseline: C101's orders, newest first

explain → SORT + COLLSCAN

2

Create the history index

idx_orders_customer_date

3

Measure again

IXSCAN • 5 keys • 5 docs • no SORT

4

A new shape: PROCESSING orders

idx_orders_fulfillment_date

5

Multikey: tags = "wireless"

idx_products_tags

6

Inventory every index

getIndexes() on all four collections

BEFORE YOU START

```
mongosh "mongodb://localhost:27017" `
  .\datasets\training_store\load.js
use training_store
```

C101 Look it up by customerNumber — don't copy an ObjectId from a slide.

GO FURTHER Steps 7–13 add catalog, unique, partial, TTL, covered, aggregation and audit practice.

KEY TAKEAWAY Record the plan shape and the counts before and after every index — that evidence is the lab.

MODULE 6 SUMMARY AND WHAT'S NEXT

From a collection scan to a small, proven set of indexes.

YOU CAN NOW

- ✓ Explain COLLSCAN, IXSCAN, FETCH and SORT
- ✓ Create, list, hide and drop named indexes
- ✓ Order compound fields with ESR and prefixes
- ✓ Choose multikey, unique, partial, sparse and TTL
- ✓ Read explain() and build a covered query
- ✓ Weigh selectivity, write cost and index usage

NEXT: MODULE 7 — REPLICATION AND SHARDING

- 1 How do replica sets keep data available?
- 2 How does sharding split a collection?
- 3 Why does a shard key need an index?

BRING ALONG Your Lab 5 index inventory and the explain() numbers you recorded before and after each index.

KEY TAKEAWAY Fast queries on one server come first — Module 7 keeps them available and spreads them across many.